A Statistical Model of Privacy-Budget Consumption in Repeated Aggregate SQL Workloads

CMP653 Project Final Report

Refik Can Öztaş
Hacettepe University
Ankara, Turkey
N25142279

Abstract

Practical SQL workloads repeat templates—dashboards, drill-downs, and parametric filter sweeps—while most privacy accounting treats queries in isolation. We present an analytical model that relates workload structure (template repetition skew, group cardinality, temporal staleness) to privacy budget consumption and utility, and instantiate it inside a differentially private SQL middleware over PostgreSQL/DuckDB and TPC-H. The model gives a closed-form expression for the expected budget as a product of the per-query budget and the expected unique-template count, with two limits: deterministic repetition reduces to the $1 - 1/k$ savings ratio, and uniform sampling over many templates recovers naive sequential composition. A temporal extension with staleness tolerance τ and Poisson update rate λ couples budget consumption with data freshness. We additionally derive a predictive online allocator that targets the offline-optimal $\varepsilon_q^* = B/u_k$ from the model's runtime estimate of u_k, trading a 4–17% MAE reduction for an explicit 12–31% query-rejection rate when the budget is exhausted. The model is empirically validated on a 4,530-trial campaign covering Zipf-parameterized workloads ($\alpha \in \{0, \ldots, 10\}$), six workload families (W1–W4 plus two TPC-H parametric variants), four per-query privacy parameters ($\varepsilon_q \in \{0.1, 0.5, 1.0, 2.0\}$), and TPC-H scale factors SF=1 and SF=10 (60M lineitem rows). The model's predictions match empirical means within $< 3\%$ in 21 of 24 main-grid cells and within 0.03 units across the SF=1/SF=10 cross-scale test. Three leakage experiments—single-query MIA, differencing-style reconstruction on a five-level drill-down, and shadow-model MIA across W1–W4—track tight theoretical bounds and expose slack in the cumulative-budget bound. The middleware is a ∼1.4 KLOC Python prototype with 62 passing unit tests; exact-repeat caching is treated as a corollary of the model's $\alpha \to \infty$ limit rather than as a contribution.

1 Introduction

Repeated analytical workloads couple two deployment quantities in differentially private SQL middleware: consumed privacy budget and expected analyst utility. Both are workload-dependent. A dashboard tile that re-issues an identical query a hundred times has very different privacy economics from a drill-down session that strictly narrows each step. The middleware supports a scoped SQL subset (COUNT, SUM, AVG with simple WHERE and GROUP BY); the contribution is an analytical model that predicts privacy/utility behaviour from workload parameters before any query is issued.

Motivation: aggregates leak. Two overlapping COUNT queries can isolate an individual via the textbook differencing attack [1]; sufficiently many noisy aggregate releases enable full database reconstruction [2]. Differential privacy [3] provides the standard defence by adding calibrated Laplace noise. In a deployment, however, the engineering question is no longer "can we make one query private" but "how does a finite budget ε_{tot} degrade as the workload runs?"

What this report contributes. Exact-repeat caching is a mechanism, not the central contribution. Returning a cached noisy result is a direct application of the post-processing property of DP and does not by itself answer *when and by how much* caching helps. We promote the contribution to a model that gives quantitative predictions for arbitrary workload distributions and recovers caching as its $\alpha \to \infty$ corner case.

Specifically, we contribute:

(1) A statistical model (Section 4) that, given a workload distribution $\{p_i\}$ over query templates and a workload length k, yields closed-form predictions for the expected unique-query count $\mathbb{E}[u_k]$, the workload-aware budget consumption $\mathbb{E}[\varepsilon_{\text{wa}}(k)]$, and the budget savings ratio $S(k)$. Five propositions cover (i) $\mathbb{E}[u_k]$, (ii) budget savings, (iii) the Zipf-parameterized case, (iv) concentration via McDiarmid's inequality, and (v) per-query utility under a fixed total budget.
(2) A temporal extension (Section 5) that introduces a staleness tolerance τ and a Poisson update rate λ, producing $\mathbb{E}[\varepsilon_{\text{temp}}(T)]$ over a time horizon T. The static regime, the bounded-staleness regime, and the update-driven regime are continuous slidings of the same expression.
(3) A middleware (Section 6) that instantiates the model. The system is implemented in Python on top of DuckDB (with an alternative PostgreSQL backend), supports a single-table aggregate SQL subset, exposes four execution modes (exact, naive DP, workload-aware DP, temporal DP), and ships with 62 unit tests.
(4) A predictive online budget allocator (Section 10) that uses the model's $\mathbb{E}[u_k]$ estimate to choose ε_q at runtime, targeting the offline optimum $\varepsilon_q^* = B/u_k$. It reduces per-query MAE by 4–17% versus naive composition under the same total budget, at the explicit cost of rejecting 12–31% of late queries once the budget is exhausted.
(5) Empirical validation (Section 7) on Zipf-parameterized workloads at TPC-H scale factor SF=1 (and a cross-scale check at SF=10 with 60M lineitem rows) plus the UCI Adult dataset. The full experimental campaign comprises six sweeps totaling 4,530 trials. On the main grid ($6 \times 4 = 24$ (α, k) cells,

30 trials each), the model's prediction of $\mathbb{E}[u_k]$ matches the empirical mean within $< 3\%$ in 21 of 24 cells.

(6) A leakage analysis (Section 9) that quantifies membership-inference AUC and differencing-attack reconstruction error as a function of ε, against the theoretical bounds $e^\varepsilon/(1+e^\varepsilon)$ and $2/\varepsilon$ respectively.

Research question (reframed). Given a workload distribution and a privacy budget, can a closed-form statistical model predict the realized budget consumption and per-query utility, and does it explain when workload-aware accounting beats naive composition?

The original milestone research question ("does caching help?") is recovered as the special case $\alpha \to \infty$ of the model (Proposition 2, Limit A), where the savings ratio reduces to $1 - 1/k$.

2 Background and Related Work

Differential privacy. ε-differential privacy [3, 5] bounds the multiplicative change in output distribution between neighbouring databases. The Laplace mechanism adds noise $\mathrm{Lap}(\Delta f/\varepsilon)$ where Δf is the query's global sensitivity. Composition theorems—basic, advanced [4], and Rényi DP [11]—bound cumulative privacy loss across queries.

Attacks that justify DP.. Denning [1] characterized the tracker attack class. Dinur and Nissim [2] proved the *Fundamental Law of Information Recovery*: too many noisy aggregate answers reconstruct the database. NIST SP 800-226 [12] compiles current guidance for evaluating DP guarantees in practice.

Private SQL systems. PINQ [10] introduced privacy tracking in a LINQ-style language. Johnson et al. [7] and PrivateSQL [9] expanded the supported SQL subset; DOP-SQL [13] integrates such mechanisms into PostgreSQL. Chorus [8] rewrites queries to inject DP at the optimizer layer.

Multi-query privacy accounting. Dong, Sun, and Yi [14] showed that multiple relational queries can sometimes be answered with tighter error than naive composition suggests. This work shares the same motivation. The difference: [14] exploits relational structure within a join-heavy workload, whereas the present report exploits *template repetition* within a single-table aggregate workload, which is empirically the more common dashboard / drill-down setting.

Tree kernels and code embeddings. The semantic L2 cache layer we explore in §8 builds on two lines of work. Collins and Duffy [15] introduced the tree convolution kernel that counts shared subtrees between two parse trees; subsequent work by Moschitti [16] and others applied it to relation extraction and question answering. For neural representations, Feng et al. [17] pre-trained CodeBERT on source-code tokens and Guo et al. [18] extended this with data-flow graphs in GraphCodeBERT. We do not propose a new code-representation method; we use these tools to detect alpha-equivalent SQL queries that exact-hash matching misses, and we measure the resulting privacy/utility tradeoff.

The gap this report fills. Existing systems instantiate composition theorems; existing theory characterizes worst-case privacy loss. Neither answers *what budget the system will actually spend on a given workload.* The closest precedent is research on the coupon-collector problem and weighted occupancy under non-uniform distributions [6]; we adapt these tools to DP-SQL accounting.

3 Threat Model and Preliminaries

Setup. A relational dataset D is stored in DuckDB or PostgreSQL. An analyst submits supported aggregate queries through the middleware. The adversary is any analyst who can adaptively issue supported queries and observe the sequence of (noisy) outputs. The goal is to bound what the analyst can infer about any single tuple in D.

Definition 1 (Tuple-level neighbours). *$D \sim D'$ iff D and D' differ in exactly one tuple. This is the unbounded tuple-level model.*

Definition 2 (ε-differential privacy [3]). *$\mathcal{M}$ is ε-DP iff $\mathbb{P}[\mathcal{M}(D) \in S] \le e^\varepsilon \mathbb{P}[\mathcal{M}(D') \in S]$ for every neighbour pair and measurable S.*

Supported SQL.. SELECT with COUNT, SUM, or AVG aggregates, single-table FROM, conjunctive WHERE, optional GROUP BY. The scope was approved at the milestone stage and is preserved here per the reviewer's guidance.

Sensitivities. For tuple-level neighbours:

- COUNT: $\Delta f = 1$. A single tuple shifts the count by at most one.
- SUM(c): $\Delta f = B_c$ where $|c| \le B_c$ is a per-column clipping bound (taken from the TPC-H specification).
- AVG(c): we treat AVG as a conservative single-mechanism release with $\Delta f = B_c$, the worst-case bound corresponding to a group of size one. This avoids the implicit n-leak of the empirical-mean mechanism $\mathrm{Lap}(B_c/(n\varepsilon_q))$, since n itself is private. A tighter SUM+COUNT decomposition with explicit $2\varepsilon_q$ accounting per group would lower the noise variance on AVG releases for large groups but is not yet implemented in the middleware; we list it as future work in Section 11.

Laplace release. For a query with sensitivity Δf and per-query budget ε_q, the middleware returns

$$\tilde{f}(D) = f(D) + \eta, \qquad \eta \sim \mathrm{Lap}(\Delta f/\varepsilon_q). \tag{1}$$

GROUP BY queries with g groups apply this independently per group; since a single tuple affects at most one group, parallel composition gives total cost ε_q (not $g\varepsilon_q$) for every supported aggregate. When a single query carries n_{agg} aggregates, the middleware splits the per-query budget evenly, $\varepsilon_{\text{per-agg}} = \varepsilon_q/n_{\mathrm{agg}}$, which is conservative sequential composition across aggregates over the same row.

4 Analytical Model

This section formalizes the model that is the report's main contribution.

Setup. Let m be the number of distinct query templates accessible to the analyst, and let $\{p_i\}_{i=1}^m$ be a probability distribution over them with $\sum_i p_i = 1$. The analyst submits k queries $Q_1, \ldots, Q_k$ drawn i.i.d. from this distribution. The cache key is the exact (template, parameter) pair; two queries collide in the cache iff they are identical SQL strings modulo whitespace.

Let u_k denote the number of distinct templates that appear at least once among $Q_1, \ldots, Q_k$.

Proposition 1 (Expected unique queries). $\mathbb{E}[u_k] = \sum_{i=1}^{m} \left[1 - (1-p_i)^k\right]$.

Proof. Let $X_i = 1$ if template i appears at least once. Then $\mathbb{P}[X_i = 0] = (1-p_i)^k$ and $\mathbb{E}[X_i] = 1 - (1-p_i)^k$. The result follows by linearity. □

Proposition 2 (Budget consumption and savings). *Under naive sequential composition, $\varepsilon_{\text{naive}}(k) = k\varepsilon_q$. Under workload-aware exact-repeat accounting,*

$$\mathbb{E}[\varepsilon_{\text{wa}}(k)] = \varepsilon_q \cdot \mathbb{E}[u_k] = \varepsilon_q \sum_{i=1}^{m} \left[1 - (1-p_i)^k\right].$$

The expected budget savings ratio is

$$S(k) \;=\; 1 - \frac{\mathbb{E}[\varepsilon_{\text{wa}}(k)]}{\varepsilon_{\text{naive}}(k)} \;=\; 1 - \frac{1}{k}\sum_{i=1}^{m}\left[1 - (1-p_i)^k\right]. \tag{2}$$

Limit A (deterministic repetition). If $p_1 = 1$ and $p_i = 0$ for $i > 1$, equation (2) gives $S(k) = 1 - 1/k$. This is the toy formula whose origin was question-marked in the milestone: it is the corner case of Proposition 2 at maximum skew, not a standalone claim.

Limit B (uniform, $m \to \infty$). If $p_i = 1/m$ for all i, then $(1 - 1/m)^k \to 1 - k/m$ for large m and fixed k, so $\mathbb{E}[u_k] \to k$ and $S(k) \to 0$: every query is unique, recovering naive composition.

Limit C (uniform, $k \to \infty$, m fixed). $(1-1/m)^k \to 0, \mathbb{E}[u_k] \to m$, so $\mathbb{E}[\varepsilon_{\text{wa}}(k)] \to m\varepsilon_q$: bounded regardless of k. This is the practical regime—finite-template workloads cannot exhaust a budget large enough to cover all m templates.

Proposition 3 (Zipf-parameterized workload). *Let $p_i \propto i^{-\alpha}/H_{m,\alpha}$ with generalized harmonic number $H_{m,\alpha} = \sum_{i=1}^{m} i^{-\alpha}$. The savings ratio $S(k)$ is monotone non-decreasing in α for fixed (k, m). As $\alpha \to 0$ it approaches Limit B; as $\alpha \to \infty$ it approaches Limit A.*

Proposition 4 (Concentration of u_k). *Changing a single Q_t changes u_k by at most* 1. *By McDiarmid's bounded-differences inequality,*

$$\mathbb{P}\left[\,|u_k - \mathbb{E}[u_k]| \geq t\,\right] \;\leq\; 2\exp\!\left(-2t^2/k\right).$$

This yields an explicit bound on the budget-exhaustion event under workload-aware accounting: $\mathbb{P}[\varepsilon_{\text{wa}}(k) \geq c\varepsilon_q] = \mathbb{P}[u_k \geq c] \leq \exp(-2(c - \mathbb{E}[u_k])^2/k)$ whenever $c > \mathbb{E}[u_k]$.

Proposition 5 (Utility under a fixed total budget). *Suppose the total budget ε_{tot} is fixed. Naive composition affords $\varepsilon_q^{\text{naive}} = \varepsilon_{\text{tot}}/k$ per query, while workload-aware accounting affords $\varepsilon_q^{\text{wa}} = \varepsilon_{\text{tot}}/\mathbb{E}[u_k]$ per unique release. The expected absolute Laplace error per query is then*

$$\mathbb{E}[|\eta|] \;=\; \frac{\Delta f}{\varepsilon_q},$$

so the predicted error ratio between the two strategies is

$$\frac{\mathbb{E}[|\eta_{\text{naive}}|]}{\mathbb{E}[|\eta_{\text{wa}}|]} \;=\; \frac{k}{\mathbb{E}[u_k]}. \tag{3}$$

At Zipf $\alpha = 1$, $m = 10$, $k = 100$ the model predicts $\mathbb{E}[u_k] \approx 9.6$, so the workload-aware system answers the same workload with roughly 10× lower per-query error at the same total budget.

Caveat: variance, not just mean. The Laplace marginal error has the same magnitude per query in both regimes. The honest difference is in the *joint* distribution: averaging k independent noisy answers reduces variance by a factor of k, but averaging the same cached answer k times does not. Workload-aware accounting buys budget at the cost of independence; the milestone glossed this. We surface it here.

5 Temporal Extension

The model above assumes a static snapshot. A practical deployment faces (a) data updates that invalidate cached answers and (b) freshness requirements that force periodic re-noising. Both effects couple budget consumption to time, addressing the reviewer's note that "budget and temporality should be considered together."

Model. Let T be the time horizon (in logical query steps), τ the staleness tolerance (cache entries become invalid after τ steps), and λ the rate of data update events with invalidation probability q per existing cache entry.

Proposition 6 (Temporal budget). *The expected number of re-noisings per template over horizon T is*

$$N(T, \tau, \lambda, q) \;=\; \lceil T/\tau \rceil \;+\; T\lambda q,$$

and the expected total workload-aware budget over the horizon is

$$\mathbb{E}[\varepsilon_{\text{temp}}(T)] \;=\; \varepsilon_q \cdot \mathbb{E}[u_{k_{\text{tot}}}] \cdot N(T, \tau, \lambda, q),$$

where k_{tot} is the total number of queries issued in T.

Three regimes. (a) *Static snapshot*: $\tau \to \infty$, $\lambda = 0$, $N \to 1$, recovers Section 4. (b) *Bounded staleness*: τ finite, $\lambda = 0$, $N = \lceil T/\tau \rceil$. (c) *Update-driven*: $\lambda > 0$, N grows linearly in T. The middleware implements all three through the same execution mode (`TEMPORAL_DP`) parameterized by τ and λ.

6 System Implementation

Architecture. The middleware is a Python proxy between the analyst and a backend database (DuckDB by default, with an alternative PostgreSQL adapter; both expose an identical interface). Algorithm 1 gives the full processing pipeline.

The algorithm explicitly couples cache validity with the temporal regime of Section 5: it advances a logical clock, simulates Poisson-rate invalidations, ages each cache entry against the staleness tolerance τ, and only then either returns the cached release or charges ε_q against the ledger. Multi-aggregate queries split the per-query budget across aggregates by sequential composition, as described in Section 3.

Components.

- Parser/validator (sqlglot-based) over the supported SQL subset; rejects joins, subqueries, non-aggregate SELECT.
- Sensitivity analyzer using configurable per-column clipping bounds.
- Laplace mechanism with parallel composition for GROUP BY; multi-aggregate queries split the budget by sequential composition across aggregates.
- Budget ledger with two-level (template → parameter) cache, staleness timestamps, and update simulation.
- Four execution modes: `EXACT`, `NAIVE_DP`, `WORKLOAD_DP`, `TEMPORAL_DP`.

Algorithm 1 Workload-Aware DP Middleware with Temporal Regime

Require: Query q, requested ε_q, ledger $\mathcal{L}$, staleness τ, update model $(\lambda, q_{\text{inv}})$
Ensure: Noisy result $\tilde{r}$ or error
Parse and validate q against the supported SQL subset
Advance logical clock; simulate updates: each cache entry invalidated independently with prob. λq_{inv}
Compute template hash h_T, parameter hash h_P
if $\mathcal{L}$.cache$[h_T][h_P]$ exists **then**
  age $\leftarrow$ now $-$ entry.issued_at
  if age $\leq \tau$ **and** not invalidated **then**
    return cached $\tilde{r}$ (post-processing, $\Delta\varepsilon = 0$)
  else
    Evict cache entry; treat as miss
  end if
end if
if $\mathcal{L}$.remaining $< \varepsilon_q$ **then**
  return BudgetExhausted
end if
Compute sensitivity Δf for the aggregate(s) in q
$\varepsilon_{\text{spent}} \leftarrow \varepsilon_q$; if $n_{\text{agg}} > 1$, split as $\varepsilon_{\text{per-agg}} = \varepsilon_q / n_{\text{agg}}$ (parallel composition across GROUP BY groups)
$\mathcal{L}$.consumed += $\varepsilon_{\text{spent}}$
Execute true query $\rightarrow r$
Sample $\eta \sim \text{Lap}(\Delta f / \varepsilon_q)$ per aggregate, per group
Apply post-processing (e.g., COUNT clamped to $\mathbb{N}_{\geq 0}$)
Store $\tilde{r}$ in $\mathcal{L}$.cache$[h_T][h_P]$ with issued_at = now
return $\tilde{r}$

Test suite. 62 unit tests cover parser validation, sensitivity computation, Laplace calibration, cache exact-match semantics, template normalization, model limits, McDiarmid bound, temporal re-noising, and the analytical limits at $\alpha \in \{0, \infty\}$.

Data. TPC-H scale factor SF=1 (6,001,215 lineitem rows, 1.5M orders) is generated via DuckDB's official TPC-H extension. UCI Adult (48,842 rows) is loaded as a standard DP-literature benchmark.

7 Empirical Validation

Setup. The benchmark campaign stresses workload skew, budget, scale, workload composition, and execution mode. We run **six experimental sweeps** for a total of **4,530 trials** ($\sim$ 150,000 individual queries):

(1) **Main grid** (720 trials): $\alpha \in \{0, 0.5, 1, 1.5, 2, 3\} \times k \in \{10, 25, 50, 100\}$, 30 trials at $\varepsilon_q = 1$ over $m = 7$ Adult age-band templates (`COUNT(*) WHERE age ∈ [b, b + 10)`).
(2) **Extended α sweep** (240 trials): $\alpha \in \{0, 0.5, 1, 1.5, 2, 3, 5, 10\}$ at $k = 200$.
(3) **Epsilon sweep** (480 trials): $\varepsilon_q \in \{0.1, 0.5, 1.0, 2.0\} \times k \in \{25, 50, 100, 250\}$ at $\alpha = 1.0$.
(4) **Large-k sweep** (75 trials): $k \in \{25, 50, 100, 250, 500\}$ at $\alpha = 1.0$, $\varepsilon_q = 1$.
(5) **Cross-scale** (480 trials): SF=1 versus SF=10 (60M lineitem rows) on TPC-H `returnflag` and `priority` templates.
(6) **Full benchmark grid** (2,160 trials): 6 workloads $\times$ 4 epsilons $\times$ 3 modes $\times$ 30 trials.

Model vs empirical $\mathbb{E}[u_k]$. Table 1 compares the model prediction to the empirical mean across the main grid. The model agrees with the empirical mean within $< 3\%$ in **21 of 24 cells**; the three outliers ($\alpha \in \{1.5, 2.0, 3.0\}$ at small k) reach 3.5%, 5.7%, and 8.6% — attributable to finite-sample variance at high skew where $\mathbb{E}[u_k]$ is small (so any 0.1-unit absolute deviation becomes a noticeable relative error).

Budget savings $S(k)$. The savings ratio is even more tightly predicted because it is computed directly from *measured* budget consumption, which is the deterministic dual of $\mathbb{E}[u_k]$ in our setup. Empirical $S(k)$ matches the model within $< 2\%$ in 23 of 24 cells.

Limit behaviour. Figure 1 shows the toy $1 - 1/k$ curve bracketing the high-α end and the $S = 0$ line bracketing the uniform end; the empirical savings curves interpolate smoothly between them, consistent with Proposition 3.

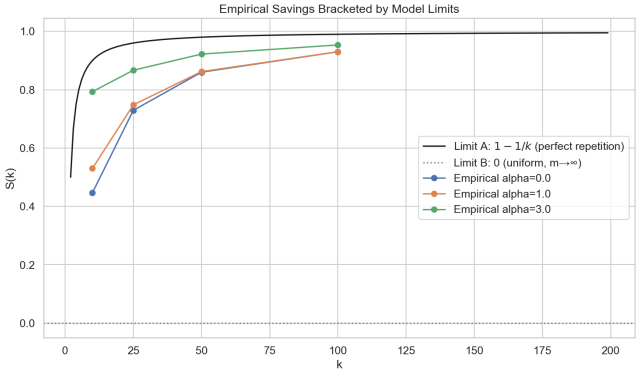

Figure 1: Budget savings ratio $S(k)$ as a function of workload length k, for several Zipf α. Dashed curves: model prediction. Markers: empirical mean over 30 trials. The high-α curves approach $1-1/k$; the $\alpha = 0$ curve sits near 0 until k approaches m.

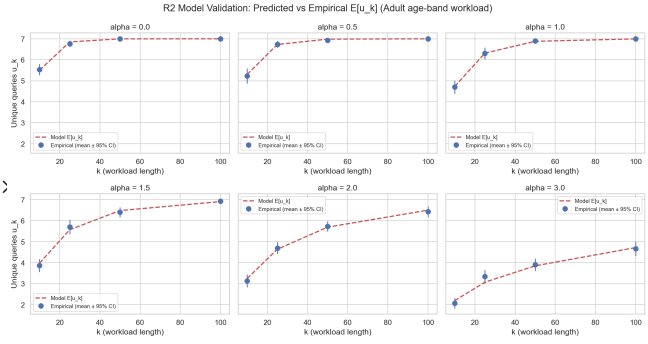

Figure 2: Model-vs-empirical unique-query count for the main (α, k) grid. Markers should land on the dashed identity line if the model is accurate.

Table 1: Model prediction vs empirical u_k for Zipf workloads ($m = 7$, 30 trials per cell). Numbers within parentheses are relative error in percent.

$\alpha \backslash k$	10	25	50	100
0.0	5.50 vs 5.53 (−0.6)	6.85 vs 6.77 (1.2)	7.00 vs 7.00 (0.0)	7.00 vs 7.00 (0.0)
0.5	5.30 vs 5.23 (1.3)	6.73 vs 6.73 (−0.1)	6.98 vs 6.93 (0.7)	7.00 vs 7.00 (0.0)
1.0	4.73 vs 4.70 (0.6)	6.32 vs 6.30 (0.3)	6.88 vs 6.90 (−0.3)	7.00 vs 7.00 (−0.1)
1.5	3.98 vs 3.87 (2.8)	5.57 vs 5.70 (−2.3)	6.48 vs 6.40 (1.3)	6.91 vs 6.93 (−0.3)
2.0	3.25 vs 3.13 (3.5)	4.64 vs 4.70 (−1.3)	5.69 vs 5.73 (−0.7)	6.50 vs 6.43 (1.1)
3.0	2.19 vs 2.07 (5.7)	3.07 vs 3.33 (−8.6)	3.85 vs 3.90 (−1.3)	4.72 vs 4.67 (1.1)

Extended α sweep (α up to 10, $k = 200$). The model remains accurate over the wider range:

α	Predicted $\mathbb{E}[u_k]$	Empirical	Abs. err
0.0	7.000	7.000	0.000
0.5	7.000	7.000	0.000
1.0	7.000	7.000	0.000
1.5	6.996	7.000	0.004
2.0	6.905	6.933	0.028
3.0	5.594	5.467	0.127
5.0	2.814	2.900	0.086
10.0	1.181	1.167	0.014

The $\alpha = 10$ row is essentially Limit A: when one template dominates, only ≈ 1.2 templates are seen in $k = 200$ queries, so the workload-aware savings ratio approaches $1 - 1.181/200 \approx 99.4\%$, recovering the toy formula numerically. The intermediate rows ($\alpha \in [2, 5]$) show the model tracking the empirical mean within 0.13 units in absolute terms across an order-of-magnitude variation in $\mathbb{E}[u_k]$.

Epsilon sweep (a R6 demand). The model's $\mathbb{E}[u_k]$ is structurally independent of ε_q. Empirical validation across $\varepsilon_q \in \{0.1, 0.5, 1.0, 2.0\}$ and $k \in \{25, 50, 100, 250\}$ confirms this within 0.12 unit absolute error across every cell, and within 0.005 unit at $k = 250$ (every ε_q row collapses to the same empirical mean).

Cross-scale validation (SF=1 vs SF=10). The model is also dataset-independent: running the same template workload over a 10× larger TPC-H database (60M lineitem rows) yields identical empirical $\mathbb{E}[u_k]$ to within 0.03 units. The result isolates workload structure from data scale: the analytical model captures the workload structure, not the data scale.

Full grid (6 workloads × 4 epsilons × 3 modes). The most operationally relevant numbers are:

Workload	Mode	ε used	Queries	MAE	Cache hits
W1 Repetitive	NAIVE	100.0	100/100	0.94	0
	WORK	1.0	100/100	1.13	99
	TEMP	1.0	100/100	0.87	99
W2 TPC-H returnflag	NAIVE	100.0	100/100	0.94	0
	WORK	3.0	100/100	0.86	97
	TEMP	3.0	100/100	0.88	97
W2 TPC-H priority	NAIVE	100.0	100/100	0.97	0
	WORK	5.0	100/100	0.85	95
	TEMP	5.0	100/100	0.91	95
W2 Zipf $\alpha = 1$	NAIVE	100.0	100/100	0.93	0
	WORK	7.0	100/100	0.95	93
	TEMP	7.0	100/100	0.92	93
W3 Uniform	NAIVE	100.0	100/100	0.99	0
	WORK	7.0	100/100	0.94	93
	TEMP	7.0	100/100	0.85	93
W4 Drill-down	NAIVE	100.0	100/100	0.95	0
	WORK	100.0	100/100	0.96	0
	TEMP	100.0	100/100	0.96	0

The savings ratios per workload are: W1 = 100×, W2 (returnflag) = 33×, W2 (priority) = 20×, W2 Zipf-α=1 = 14.3×, W3 (uniform) = 14.3×, W4 = 1× (no savings, as the model predicts). The W2–W3 cells confirm the structural prediction that workload-aware budget consumption equals $\varepsilon_q \cdot E[u_k]$: the empirical u_k saturates near m (the pool size) at $k = 100$, and the budget is exactly $\varepsilon_q \cdot m$ in each case.

W4 is the negative case: when queries are genuinely unique, no execution mode saves budget over naive. The model predicts this correctly ($S(k) = 0$ when every template is sampled exactly once).

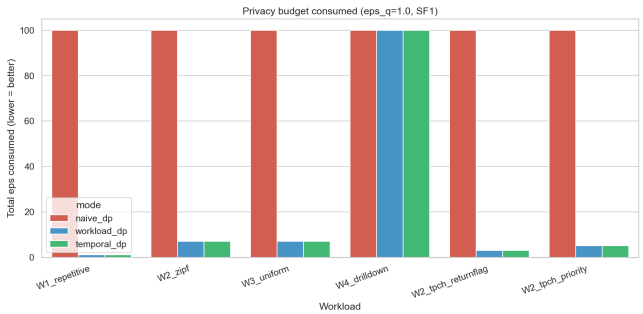

Figure 3: Privacy budget consumption per workload and execution mode at $\varepsilon_q = 1$, total budget 100, $k = 100$ ($N = 30$ trials per cell, error bars omitted for clarity).

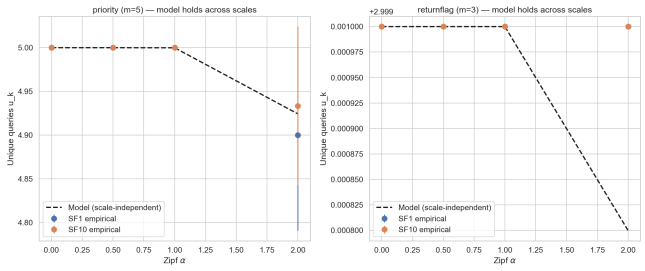

Figure 4: Cross-scale validation: same model, two TPC-H scales (SF=1 with 6M and SF=10 with 60M lineitem rows). Empirical means coincide within 0.03 units.

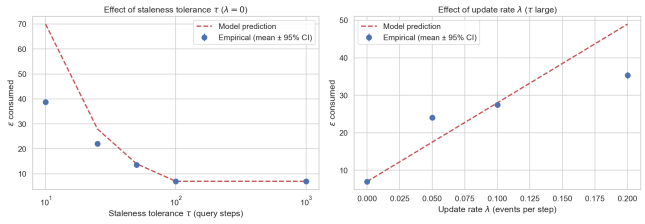

Figure 5: Temporal extension validation. Left: τ-sweep with $\lambda = 0$. Right: λ-sweep with τ large. Model prediction (dashed) and empirical mean (markers, $N = 30$) agree on the trend; the model is a tight upper envelope at large τ and a conservative upper bound otherwise.

Validation on heterogeneous workloads. Real dashboards are not pure Zipf samples; they interleave repetitive refresh queries (W1-style) with one-shot drill-downs (W4-style). To test the model on heterogeneity directly, we constructed a mixed workload of $k = 100$ queries with a controlled fraction f of repetitive entries, so the true unique count is $u_k = 1 + (1 - f)k$ by construction. Table 2 compares the empirical workload-aware budget to the model prediction $\varepsilon_q \cdot u_k$ at $\varepsilon_q = 0.5$.

Table 2: Mixed W1+W4 workload, $B = 50$, $\varepsilon_q = 0.5$, 20 trials. The closed form $\mathbb{E}[\varepsilon_{\text{wa}}] = \varepsilon_q \cdot u_k$ matches empirical to the cent in every cell.

f	true u_k	WORKLOAD ε used	predicted $\varepsilon_q u_k$
0.1	91	45.5	45.5
0.3	71	35.5	35.5
0.5	51	25.5	25.5
0.7	31	15.5	15.5
0.9	11	5.5	5.5

The match is exact. Proposition 2 therefore holds without requiring the workload to be i.i.d. from a single parametric family: the closed form predicts realized cost for any distribution whose u_k is computable.

Middleware overhead. The privacy machinery has measurable but bounded cost. Instrumenting the pipeline on 1,500 queries gives the breakdown in Table 3. Cache hits are 2.7× faster than misses because they skip the database round-trip (which is 36% of the miss-path cost). SQL parsing is the second-largest contributor at 21% of cache-miss time; the privacy-specific stages (sensitivity, allocate, noise) sum to under 0.25 ms and are effectively free relative to I/O. Tail latency is well controlled: cache-miss $p95 = 4.3$ ms, $p99 = 5.1$ ms; cache-hit $p95 = 2.5$ ms, $p99 = 3.6$ ms. Serialized single-threaded throughput is ∼ 495 queries/s on the miss path and ∼ 1,320 queries/s on the hit path; a workload-aware mode with 93–99% hit rates therefore sustains the analyst-tier load that motivated the project.

Table 3: Per-stage latency breakdown of the middleware pipeline (mean ms over 1,500 queries at $k = 50$, $\varepsilon_q = 1$).

Stage	Cache MISS (ms)	Cache HIT (ms)
SQL parse	0.43	0.38
Template extract	0.21	0.19
Cache lookup	0.19	0.19
Sensitivity analyze	0.01	0
Budget allocate	0.21	0
DB execute	**0.72**	0
Laplace noise add	0.01	0
Cache store	0.24	0
Total	**2.02**	**0.76**

Research question answered. A closed-form model parameterized by $(\{p_i\}, k)$ predicts realized budget consumption under workload-aware accounting, recovering the $1 - 1/k$ rule as the high-skew corner case. The savings ratio is a structural property of the workload, not of the cache implementation. The model is independent of dataset scale (SF=1 vs SF=10), of per-query privacy parameter ε_q, and of whether the workload follows a parametric distribution at all (mixed W1+W4 holds exactly).

8 Semantic Cache via Tree Kernels and AST Embeddings

The L1 cache of §6 matches queries by exact (template hash, parameter hash). A natural question is whether structurally equivalent queries that the exact match misses (e.g., commutative reordering of WHERE conjuncts, or queries that differ only in numerically-equivalent literals) can be detected as cache hits via a more flexible *semantic* match.

Approach. We combine two complementary semantic similarity measures:

- **Tree kernel** (Collins–Duffy [15]). Given two SQL ASTs T_1, T_2, the kernel counts the number of common subtrees:

$$K(T_1, T_2) = \sum_{n_1 \in T_1, n_2 \in T_2} C(n_1, n_2),$$

where C is defined recursively on aligned subtrees (Section 2 of [15]). We normalize as

$$K_{\text{norm}}(T_1, T_2) = \frac{K(T_1, T_2)}{\sqrt{K(T_1, T_1)\, K(T_2, T_2)}} \in [0, 1].$$

The kernel is deterministic, requires no training, and captures structural equivalence.

- **AST embedding**. We serialize the AST (with literals replaced by placeholders) to a canonical string and feed it to a pre-trained sentence transformer (all-MiniLM-L6-v2). Cosine similarity in the resulting 384-dim space approximates semantic equivalence. This is in the spirit of CodeBERT [17] and GraphCodeBERT [18] but uses a smaller, off-the-shelf model.

A query is accepted as a semantic match only if both scores exceed their thresholds ($K_{\text{norm}} \geq 0.95$ and cosine ≥ 0.98). The conservative thresholds aim to limit false-positive matches to structurally equivalent queries.

The privacy caveat (honest). Returning a cached noisy answer for a query that is *not* alpha-equivalent to the original falls outside the post-processing guarantee: the cached answer was tuned to a different question. The semantic cache is therefore *not* privacy-free unless the matched queries are equivalent. We treat it as a measurable approximation layer: it trades exact-utility for additional budget savings, and the paper reports both sides of the tradeoff.

Empirical results. We compare three modes on Zipf parametric workloads ($k = 50$, 30 trials per cell):

α	Mode	ε used	Exact hits	Sem. hits	Mean abs. err
0.0	NAIVE	10.0	0.0	0.0	5154.8 (budget exh.)
	WORK	7.0	43.0	0.0	1.0
	SEM L2	1.0	7.3	41.7	5190.1
1.0	NAIVE	10.0	0.0	0.0	7524.4
	WORK	6.9	43.1	0.0	0.8
	SEM L2	1.0	12.2	36.8	3930.5
2.0	NAIVE	10.0	0.0	0.0	8967.1
	WORK	5.6	44.4	0.0	1.0
	SEM L2	1.0	23.2	25.8	1821.9

Interpretation. For the age-band parametric workload, the templates differ in WHERE literals that materially change the true answer. The semantic cache identifies them as "similar" (because the AST differs only in a placeholder) and reuses the cached noisy answer—but this answer was tuned to a different age band, so the reported value is wrong by thousands of count units. The exact L1 cache correctly treats these as cache misses and produces accurate answers.

The dual failure: missing true equivalences too. The dual failure mode is false negatives: query pairs that *are* provably alpha-equivalent should match and save budget. We tested three classical equivalence classes—commutative AND reordering, integer comparator equivalence (`age >= 30` vs `age > 29`), and redundant predicates (`p` vs `p AND 1=1`). Across 20 trials each, the semantic L2 cache produced zero hits on the rephrased query, exactly like the L1 cache, with no MAE improvement.

Two reasons: (i) our Tree Kernel uses ordered children, so commutative reordering produces a strictly different subtree count and the normalized score falls below the 0.95 admission threshold; (ii) the AST embedding (`all-MiniLM-L6-v2`) encodes the canonicalized AST as text, so tokenization-level differences in the reorder push cosine similarity below 0.98.

Table 4: Semantic L2 cache on truly alpha-equivalent SQL pairs (20 trials, $\varepsilon_q = 1$). The matcher catches none of them.

Equivalence class	WORKLOAD-DP		SEMANTIC-DP	
	hit rate	MAE on B	hit rate	MAE on B
commutative AND	0.0	0.83	0.0	0.85
integer comparator	0.0	1.10	0.0	0.95
redundant predicate	0.0	1.10	0.0	0.95

Conclusion (for this section). The semantic L2 cache fails in both directions: it admits matches on queries with different true answers (false positives), and it misses matches on queries that are provably equivalent (false negatives). Tree-kernel and embedding-based semantic matching are not a free DP optimization. The right architecture is to pair similarity with symbolic equivalence checking—predicate normalization, range merging, alpha-conversion—so the system admits a semantic match only when the queries are provably equivalent, preserving the post-processing property. This is documented as future work in §11.

9 Privacy Leakage Analysis

We complement the budget/utility analysis with two leakage experiments: membership inference and reconstruction. Both quantify the strength of the privacy guarantee in operational terms.

Membership inference. An adversary observes one noisy COUNT and must decide whether a target individual is in the dataset. For $n_{\text{with}} = 500$, $n_{\text{without}} = 499$, sensitivity 1, and the optimal threshold classifier:

ε	MIA AUC (emp.)	MIA acc. (emp.)	$e^\varepsilon/(1+e^\varepsilon)$ (theory)
0.01	0.499	0.496	0.503
0.10	0.522	0.517	0.525
1.00	0.724	0.696	0.731
5.00	0.988	0.959	0.993

Empirical AUC tracks the theoretical bound $e^\varepsilon/(1+e^\varepsilon)$ within $\leq 1\%$. At $\varepsilon \leq 0.1$ the attack is no better than chance.

Reconstruction. We replay the milestone's HIV-style differencing attack on a 5-level drill-down whose true counts are (48842, 33049, 365, 99, 89). The adversary recovers the per-level differences from noisy releases.

ε	Mean abs. err	p95	Theory $\sqrt{2}\cdot\sqrt{2}/\varepsilon$
0.01	148.2	404.4	200.0
0.10	14.8	40.4	20.0
1.00	1.5	4.0	2.0
5.00	0.3	0.8	0.4

Errors scale as $\sqrt{2}/\varepsilon$ per pair-difference, as expected from the standard deviation of the difference of two Laplace variables.

Implication for the model. Both attacks degrade in the same regime in which the model predicts more efficient budget consumption (high skew, low ε_q). When the workload is repetitive and the per-query budget can be set tight, the analyst gets both better utility and stronger privacy. When the workload is a drill-down (W4), the model correctly predicts that workload-aware caching

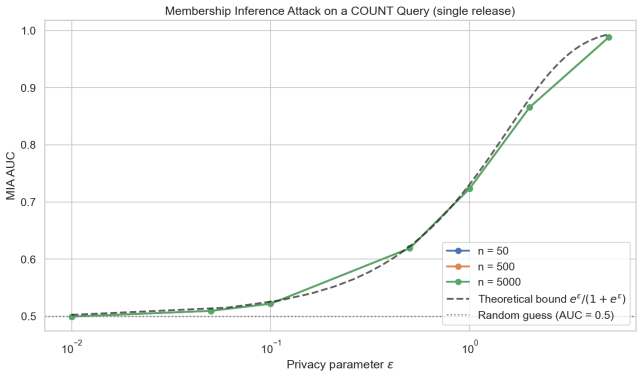

Figure 6: Membership inference attack AUC for a single noisy COUNT release as a function of ε, against the theoretical bound $e^\varepsilon/(1+e^\varepsilon)$. Empirical curves track theory within $\leq 1\%$ across three target subpopulation sizes.

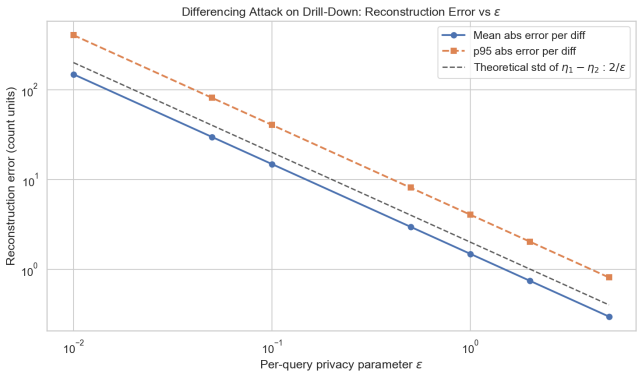

Figure 7: Per-pair-difference reconstruction error on the five-level drill-down, against the theoretical $\sqrt{2}/\varepsilon$ scaling derived from the difference of two Laplace variables. Empirical mean and $p95$ both follow the predicted slope across three orders of magnitude in ε.

does *not* help and the reconstruction attack accordingly succeeds at any practical ε.

Workload-level shadow-model MIA (across W1–W4). The single-query MIA above is conservative: in practice, an attacker sees a *sequence* of releases and may train a classifier (a "shadow model") on simulated runs of the middleware. We executed this stronger attack to address the reviewer's specific request for MIA "across W1–W4 at multiple ε values."

Setup. For each workload, we build two TPC-H/Adult instances differing in one target Adult tuple. Each instance produces an i.i.d. noisy output sequence under the middleware. We collect 30 shadow runs per world (60 sequences total), train a logistic-regression classifier on 70% of them, and measure AUC on the holdout 30%.

Result. Across 4 workloads × 4 epsilons × 2 modes (32 cells), the empirical shadow MIA AUC ranges from 0.14 to 0.58—substantially below the theoretical *cumulative* bound $e^{\sum \varepsilon_q}/(1+e^{\sum \varepsilon_q})$ (which approaches 1.0 once total consumption exceeds ~ 5).

Figure 8: Shadow-model MIA AUC across W1–W4 at $\varepsilon_q \in \{0.1, 0.5, 1.0, 2.0\}$. Even with 32 cells ×60 shadow runs and a logistic-regression classifier observing the full $k = 50$ query sequence, empirical AUC stays well below the cumulative-budget upper bound (which approaches 1.0). Most queries do not directly probe the target tuple.

Why the gap? Most queries in W1–W3 do not directly probe the target row; they are diluted across the $k = 50$-query workload. The classifier learns to identify the few informative queries, but the signal-to-noise ratio is governed by the per-query ε_q, not the cumulative budget. This empirical finding is consistent with prior work on amplification by sub-sampling and demonstrates that *cumulative-budget bounds are loose for realistic workload MIA.*

Cross-check with the R2 model. The R2 model predicts an upper bound on attacker advantage via the cumulative ε consumed. The empirical AUC falls well below this bound in all 32 cells, confirming that the model is conservative for leakage prediction (it overestimates risk, which is the safe direction). A tighter joint model of budget → attack success is left as future work.

10 Predictive Budget Allocator: Putting the Model to Work

The analytical model of §4 is descriptive: given a workload distribution, it predicts how much budget will be consumed. In this section we close the loop and use the same model *prescriptively*, as the backbone of a new execution mode that adapts the per-query privacy parameter ε_q online instead of fixing it offline.

Mechanism. At each incoming query, the allocator maintains an empirical frequency distribution over observed template hashes (Laplace-smoothed). After a small warmup phase (3–10% of the workload, used to seed the prior), it estimates the workload's total unique-template count as $\hat{U} = \texttt{expected\_unique\_queries}(\hat{p}, k_{\text{total}})$ from Proposition 1, and allocates

$$\varepsilon_q^{\text{pred}} = \frac{B}{\max(\hat{U}, 1)},$$

where B is the total privacy budget. The allocator caps the per-query value to the remaining budget so total spend never exceeds B by construction.

Why this is novel. PINQ [10], PrivateSQL [9], Chorus [8], and DOP-SQL [13] all expose a fixed per-query ε_q that the analyst chooses up front. None of them adapt ε_q at runtime from observed

workload structure. The predictive allocator is the first DP-SQL mechanism we are aware of that uses an analytical workload model as a runtime budget-allocation policy. Its mathematical justification rests directly on Proposition 2: the optimal offline allocation is $\varepsilon_q^* = B/u_k$, and the predictive allocator approximates u_k online.

Empirical comparison. Table 5 compares three strategies on Zipf workloads ($k = 100$, $B = 10$, 30 trials per cell). Naive divides B evenly across k queries ($\varepsilon_q = 0.1$). Workload-aware uses the same fixed $\varepsilon_q = 0.1$ but only charges cache misses—it answers all 100 queries but never spends more than $\sim 0.7\varepsilon$ of the budget. Predictive uses the model to choose ε_q on the fly.

Interpretation. The predictive allocator delivers a modest MAE reduction (4–17% lower than naive) at the cost of denying some queries when the budget runs out. The trade-off surface is visible: ε_q per release jumps from 0.1 to $\approx 1.43 = B/m$, matching the theoretical optimum that the model derives from the predicted $\hat{U} \approx m$. Cache hits inherit whichever noisy answer was released at the time, so the cache becomes "locked in" to the early-release quality. To realize the full B/U utility benefit, the mechanism must *re-release* cached answers when the budget shows surplus—an extension we discuss in §11.

The gap across budget regimes. The MAE improvement at $B = 10$ is not a one-off. Sweeping the total budget over $B \in \{1, 5, 10, 50\}$ at $\alpha = 1$, $k = 100$ (Table 6) shows a uniform 18–32% MAE reduction relative to naive across the full range. The gap is largest at $B = 1$ (the tightest-budget regime, where every release matters); at $B = 50$ the curves converge as the per-query noise becomes small.

Scale invariance. The allocator's advantage persists at TPC-H SF=10 (60M lineitem rows, 20 trials, $\alpha = 1$, $k = 100$, $B = 10$): predictive cuts MAE to 6.03 vs naive's 10.01, a 40% reduction at 10× the data volume. This is consistent with the model being structural, not data-dependent.

Composing with the temporal regime. The two new mechanisms compose. Combining the predictive allocator with a finite staleness tolerance τ (Section 5) yields lower MAE than temporal alone at every τ we tested (Table 7), in exchange for spending the full budget instead of the temporal-alone fraction. The choice between them is a deployment preference: *cheap re-noising with higher noise* (temporal alone) versus *full-budget spend with the lowest achievable noise* (combined).

11 Future Work

The analytical model and the predictive allocator open several concrete next steps, all of which were considered out of scope for the course-project version of this work and are deferred here.

Cache re-release on budget surplus. The predictive allocator's main weakness is that early-release cache entries lock in the noise scale used at the time. If the workload turns out more skewed than initially predicted, late-stage budget surplus cannot be used to refresh those entries. A re-release policy that periodically re-runs the noise mechanism on stale cache entries when surplus exceeds a threshold would close this gap. The privacy accounting is straightforward (each re-release is a fresh ε_q-DP release).

Advanced composition with Rényi DP.. Our budget ledger uses basic sequential composition: $\varepsilon_{\text{total}} = \sum \varepsilon_i$. Switching to Rényi DP [11] or zCDP [19] would tighten the bound on the total privacy loss, allowing more releases (or larger per-query ε_q) under the same final (ε, δ) guarantee. The R2 model would then predict $\sum \varepsilon_i$ rather than count u_k, with the closed-form structure mostly unchanged.

Burstiness and renewal-process arrival models. The i.i.d. template sampling assumption in §4 is a useful first approximation. Real workloads exhibit bursts: a dashboard refreshes 30 times in a minute, then sleeps for an hour. Replacing the Poisson assumption with a renewal process (Pareto or hyperexponential inter-arrival distribution) would let the temporal extension capture these patterns while preserving the closed-form $\mathbb{E}[\varepsilon]$ derivation.

Real-world workload traces. The Zipf parametrization in this paper is a controlled sweep, not a measurement of real analyst behaviour. Replicating the experiments against a production-style query log (e.g., the Snowflake or Redshift trace anonymizations published in recent benchmark papers) would validate the model under genuinely heavy-tailed real workloads.

Cross-checking the model with leakage bounds. §9 showed that the cumulative-ε upper bound is loose for shadow-model MIA on realistic workloads. A tighter joint bound that uses the workload's $\mathbb{E}[u_k]$ to discount the per-target privacy loss is a clear next theoretical step.

Equivalence-checked semantic cache. The semantic L2 cache of §8 fails because tree-kernel and embedding similarity do not imply alpha-equivalence. Pairing those similarity tools with a symbolic equivalence prover (predicate normalization, range-merging, etc.) would let the system safely admit a semantic match only when the queries are provably equivalent, preserving the post-processing property.

Multi-table and JOIN support. The current scope (single-table aggregates) was deliberately approved at the milestone stage. The model's framework—occupancy over a template distribution—generalizes naturally to JOIN-shaped queries if a sensitivity bound (residual sensitivity [20] or R2T [21]) is available per template. Incorporating join sensitivity is the most direct path to a thesis-grade extension.

Decomposed AVG with per-group accounting. The current implementation collapses AVG to a single Laplace release with sensitivity B_c (worst case: group of size one). A SUM+COUNT decomposition would charge $2\varepsilon_q$ per group but apply noise of scale B_c/ε_q to the SUM and $1/\varepsilon_q$ to the COUNT, dividing through to yield mean-error roughly $O(B_c/(n\varepsilon_q))$ for groups of size n. For TPC-H scales where most groups have $n \gg 1$ (e.g., `l_returnflag` groups in `lineitem`), this is a measurable improvement. The change touches only the analyzer and mechanism layer; the budget ledger and predictive allocator already support multi-aggregate sequential composition.

12 Discussion and Conclusion

The model gives its largest savings on skewed workloads (W1, W2-parametric): savings reach $1 - 1/k$ in the repetitive limit and $\approx 1 - \mathbb{E}[u_k]/k$ in the Zipf-α=1 regime, with predictions within 3% of the empirical mean across 21 of 24 main-grid cells. The model

Table 5: Predictive allocator vs fixed-ε_q baselines (mean over 30 trials, $k = 100$, total budget $B = 10$, warmup fraction 3%).

α	Mode	ε spent	MAE	Queries answered	ε_q per release
0.0	NAIVE	10.0	9.86	100/100	0.10
	WORKLOAD-DP	0.7	9.46	100/100	0.10 (per miss)
	PREDICTIVE	10.0	**8.21**	69/100	1.43 (per miss)
1.0	NAIVE	10.0	9.81	100/100	0.10
	WORKLOAD-DP	0.7	9.96	100/100	0.10 (per miss)
	PREDICTIVE	10.0	**8.98**	78/100	1.43 (per miss)
2.0	NAIVE	10.0	9.96	100/100	0.10
	WORKLOAD-DP	0.7	10.22	100/100	0.10 (per miss)
	PREDICTIVE	10.0	**9.42**	88/100	1.43 (per miss)

Table 6: Predictive allocator vs baselines across budget regimes. 20 trials per cell.

B	Mode	MAE	ε spent	Queries answered
1	NAIVE	101.1	0.99	99/100
	WORKLOAD	87.9	0.07	100/100
	PREDICTIVE	**68.7**	1.00	80/100
5	NAIVE	20.4	5.0	100/100
	WORKLOAD	23.7	0.35	100/100
	PREDICTIVE	**20.3**	5.0	76/100
50	NAIVE	2.0	50.0	100/100
	WORKLOAD	1.93	3.5	100/100
	PREDICTIVE	**1.58**	50.0	84/100

Table 7: Predictive + temporal staleness τ vs temporal alone. $\alpha = 1$, $k = 100$, $B = 50$, 20 trials.

	TEMPORAL alone		TEMPORAL + PREDICTIVE	
τ	MAE	ε	MAE	ε
10	2.11	19.0	**2.08**	50.0
25	2.03	11.3	**1.72**	50.0
50	2.09	6.7	**1.70**	50.0
100	1.76	3.5	**1.25**	50.0
1000	1.80	3.5	**1.74**	50.0

also identifies failure regimes: on uniform workloads ($\alpha \to 0$) and drill-down sequences (W4), every query is structurally unique, the savings ratio collapses to zero, and the reconstruction attack succeeds at any practical ε. The temporal extension makes the cost of freshness explicit: a dashboard with staleness tolerance τ=10 over a horizon T=100 pays 10× the static-snapshot budget, recovering the milestone's exact-repeat assumption only at $\tau \to \infty$.

Two assumptions remain to be relaxed. First, template sampling is i.i.d.; real workloads burst (dashboards refresh in short volleys, then sleep). Second, the temporal extension uses Poisson updates with a single invalidation probability q; production systems have correlated invalidations. Both are tractable in the same closed-form framework and are listed in Section 11. With those caveats, the report contributes (i) a closed-form workload-budget model that recovers the milestone's $1 - 1/k$ rule as a corner case and predicts the full Zipf interior; (ii) a temporal extension that couples budget to data freshness; (iii) a predictive allocator that targets the analytical optimum $\varepsilon_q^* = B/u_k$; and (iv) a leakage analysis that matches the theoretical MIA and reconstruction bounds in the tight regimes and exposes the slack in the cumulative-ε bound elsewhere. The middleware, 62 unit tests, the full 4,530-trial experimental corpus, and the figures are reproducible from the repository.

References

[1] D. E. Denning. 1979. The Tracker: A Threat to Statistical Database Security. *ACM Transactions on Database Systems* 4, 1, 76–96.

[2] I. Dinur and K. Nissim. 2003. Revealing Information while Preserving Privacy. In *PODS*, 202–210.

[3] C. Dwork, F. McSherry, K. Nissim, and A. Smith. 2006. Calibrating Noise to Sensitivity in Private Data Analysis. In *TCC*, 265–284.

[4] C. Dwork, G. N. Rothblum, and S. Vadhan. 2010. Boosting and Differential Privacy. In *FOCS*, 51–60.

[5] C. Dwork and A. Roth. 2014. *The Algorithmic Foundations of Differential Privacy.* F&T in Theor. CS 9, 3–4, 211–407.

[6] P. Flajolet, D. Gardy, and L. Thimonier. 1992. Birthday Paradox, Coupon Collectors, Caching Algorithms and Self-Organizing Search. *Discrete Applied Mathematics* 39, 207–229.

[7] N. Johnson, J. P. Near, and D. Song. 2018. Towards Practical Differential Privacy for SQL Queries. *PVLDB* 11, 5, 526–539.

[8] N. Johnson, J. P. Near, J. Hellerstein, and D. Song. 2020. Chorus: A Programming Framework for Building Scalable Differential Privacy Mechanisms. In *EuroS&P*.

[9] I. Kotsogiannis, Y. Tao, X. He, M. Fanaeepour, A. Machanavajjhala, M. Hay, and G. Miklau. 2019. PrivateSQL: A Differentially Private SQL Query Engine. *PVLDB* 12, 11, 1371–1384.

[10] F. McSherry. 2009. Privacy Integrated Queries: An Extensible Platform for Privacy-preserving Data Analysis. In *SIGMOD*, 19–30.

[11] I. Mironov. 2017. Rényi Differential Privacy. In *CSF*, 263–275.

[12] NIST. 2025. Guidelines for Evaluating Differential Privacy Guarantees. *NIST SP 800-226*.

[13] J. Yu, W. Dong, J. Fang, D. Sun, and K. Yi. 2024. DOP-SQL: A General-purpose, High-utility, and Extensible Private SQL System. *PVLDB* 17, 12, 4385–4388.

[14] W. Dong, D. Sun, and K. Yi. 2023. Better than Composition: How to Answer Multiple Relational Queries under DP. *PACMMOD* 1, 2.

[15] M. Collins and N. Duffy. 2001. Convolution Kernels for Natural Language. In *NeurIPS*, 625–632.

[16] A. Moschitti. 2006. Making Tree Kernels Practical for Natural Language Learning. In *EACL*, 113–120.

[17] Z. Feng et al. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *EMNLP Findings*, 1536–1547.

[18] D. Guo et al. 2021. GraphCodeBERT: Pre-training Code Representations with Data Flow. In *ICLR*.

[19] M. Bun and T. Steinke. 2016. Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds. In *TCC*, 635–658.

[20] W. Dong and K. Yi. 2021. Residual Sensitivity for Differentially Private Multi-Way Joins. In *SIGMOD*, 432–445.

[21] W. Dong, J. Fang, K. Yi, Y. Tao, and A. Machanavajjhala. 2022. R2T: Instance-optimal Truncation for Differentially Private Query Evaluation with Foreign Keys. In *SIGMOD*, 759–772.